



# Troubleshooting Clusters

## Troubleshooting Clusters

Debugging common cluster issues.

This doc is about cluster troubleshooting; we assume you have already ruled out your application as the root cause of the problem you are experiencing. See the [application troubleshooting guide](#) for tips on application debugging. You may also visit the [troubleshooting overview document](#) for more information.

For troubleshooting kubectl, refer to [Troubleshooting kubectl](#).

## Listing your cluster

The first thing to debug in your cluster is if your nodes are all registered correctly.

Run the following command:

```
kubectl get nodes
```

And verify that all of the nodes you expect to see are present and that they are all in the `Ready` state.

To get detailed information about the overall health of your cluster, you can run:

```
kubectl cluster-info dump
```

## Example: debugging a down/unreachable node

Sometimes when debugging it can be useful to look at the status of a node -- for example, because you've noticed strange behavior of a Pod that's running on the node, or to find out why a Pod won't schedule onto the node. As with Pods, you can use

`kubectl describe node` and `kubectl get node -o yaml` to retrieve detailed information about nodes. For example, here's what you'll see if a node is down (disconnected from the network, or kubelet dies and won't restart, etc.). Notice the events that show the node is NotReady, and also notice that the pods are no longer running (they are evicted after five minutes of NotReady status).

```
kubectl get nodes
```

| NAME                 | STATUS   | ROLES  | AGE | VERSION |
|----------------------|----------|--------|-----|---------|
| kube-worker-1        | NotReady | <none> | 1h  | v1.23.3 |
| kubernetes-node-bols | Ready    | <none> | 1h  | v1.23.3 |
| kubernetes-node-st6x | Ready    | <none> | 1h  | v1.23.3 |
| kubernetes-node-unaj | Ready    | <none> | 1h  | v1.23.3 |

```
kubectl describe node kube-worker-1
```

```
Name: kube-worker-1
Roles: <none>
Labels: beta.kubernetes.io/arch=amd64
        beta.kubernetes.io/os=linux
        kubernetes.io/arch=amd64
        kubernetes.io/hostname=kube-worker-1
        kubernetes.io/os=linux
        node.alpha.kubernetes.io/ttl: 0
        volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp: Thu, 17 Feb 2022 16:46:30 -0500
Taints: node.kubernetes.io/unreachable:NoExecute
        node.kubernetes.io/unreachable:NoSchedule
Unschedulable: false
Lease:
  HolderIdentity: kube-worker-1
  AcquireTime: <unset>
  RenewTime: Thu, 17 Feb 2022 17:13:09 -0500
Conditions:
  Type           Status      LastHeartbeatTime      LastTransitionTime
  ----           -
  NetworkUnavailable  False      Thu, 17 Feb 2022 17:09:13 -0500  Thu, 17 Feb 2022 17:09:13 -0500
  MemoryPressure      Unknown    Thu, 17 Feb 2022 17:12:40 -0500  Thu, 17 Feb 2022 17:12:40 -0500
  DiskPressure        Unknown    Thu, 17 Feb 2022 17:12:40 -0500  Thu, 17 Feb 2022 17:12:40 -0500
  PIDPressure         Unknown    Thu, 17 Feb 2022 17:12:40 -0500  Thu, 17 Feb 2022 17:12:40 -0500
  Ready               Unknown    Thu, 17 Feb 2022 17:12:40 -0500  Thu, 17 Feb 2022 17:12:40 -0500
```

```

Addresses:
  InternalIP: 192.168.0.113
  Hostname:   kube-worker-1
Capacity:
  cpu:                2
  ephemeral-storage: 15372232Ki
  hugepages-2Mi:      0
  memory:             2025188Ki
  pods:              110
Allocatable:
  cpu:                2
  ephemeral-storage: 14167048988
  hugepages-2Mi:      0
  memory:             1922788Ki
  pods:              110
System Info:
  Machine ID:          9384e2927f544209b5d7b67474bbf92b
  System UUID:         aa829ca9-73d7-064d-9019-df07404ad448
  Boot ID:             5a295a03-aaca-4340-af20-1327fa5dab5c
  Kernel Version:      5.13.0-28-generic
  OS Image:            Ubuntu 21.10
  Operating System:    linux
  Architecture:        amd64
  Container Runtime Version: containerd://1.5.9
  Kubelet Version:     v1.23.3
  Kube-Proxy Version:  v1.23.3
Non-terminated Pods:  (4 in total)
  Namespace           Name                                           CPU Requests  CPU Limits
  -----
  default              nginx-deployment-67d4bdd6f5-cx2nz           500m (25%)    500m (25%)
  default              nginx-deployment-67d4bdd6f5-w6kd7           500m (25%)    500m (25%)
  kube-system          kube-proxy-dnxbz                             0 (0%)        0 (0%)
  kube-system          weave-net-gjxxp                             100m (5%)     0 (0%)
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests      Limits
  -----
  cpu                 1100m (55%)  1 (50%)
  memory              456Mi (24%)  256Mi (13%)
  ephemeral-storage   0 (0%)       0 (0%)
  hugepages-2Mi       0 (0%)       0 (0%)
Events:
  ...

```

```
kubectl get node kube-worker-1 -o yaml
```

```
apiVersion: v1
kind: Node
metadata:
  annotations:
    node.alpha.kubernetes.io/ttl: "0"
    volumes.kubernetes.io/controller-managed-attach-detach: "true"
  creationTimestamp: "2022-02-17T21:46:30Z"
  labels:
    beta.kubernetes.io/arch: amd64
    beta.kubernetes.io/os: linux
    kubernetes.io/arch: amd64
    kubernetes.io/hostname: kube-worker-1
    kubernetes.io/os: linux
  name: kube-worker-1
  resourceVersion: "4026"
  uid: 98efe7cb-2978-4a0b-842a-1a7bf12c05f8
spec: {}
status:
  addresses:
    - address: 192.168.0.113
      type: InternalIP
    - address: kube-worker-1
      type: Hostname
  allocatable:
    cpu: "2"
    ephemeral-storage: "14167048988"
    hugepages-2Mi: "0"
    memory: 1922788Ki
    pods: "110"
  capacity:
    cpu: "2"
    ephemeral-storage: 15372232Ki
    hugepages-2Mi: "0"
    memory: 2025188Ki
    pods: "110"
  conditions:
    - lastHeartbeatTime: "2022-02-17T22:20:32Z"
      lastTransitionTime: "2022-02-17T22:20:32Z"
      message: Weave pod has set this
      reason: WeaveIsUp
      status: "False"
      type: NetworkUnavailable
    - lastHeartbeatTime: "2022-02-17T22:20:15Z"
      lastTransitionTime: "2022-02-17T22:13:25Z"
      message: kubelet has sufficient memory available
      reason: KubeletHasSufficientMemory
```

```

status: "False"
type: MemoryPressure
- lastHeartbeatTime: "2022-02-17T22:20:15Z"
  lastTransitionTime: "2022-02-17T22:13:25Z"
  message: kubelet has no disk pressure
  reason: KubeletHasNoDiskPressure
  status: "False"
  type: DiskPressure
- lastHeartbeatTime: "2022-02-17T22:20:15Z"
  lastTransitionTime: "2022-02-17T22:13:25Z"
  message: kubelet has sufficient PID available
  reason: KubeletHasSufficientPID
  status: "False"
  type: PIDPressure
- lastHeartbeatTime: "2022-02-17T22:20:15Z"
  lastTransitionTime: "2022-02-17T22:15:15Z"
  message: kubelet is posting ready status
  reason: KubeletReady
  status: "True"
  type: Ready
daemonEndpoints:
  kubeletEndpoint:
    Port: 10250
nodeInfo:
  architecture: amd64
  bootID: 22333234-7a6b-44d4-9ce1-67e31dc7e369
  containerRuntimeVersion: containerd://1.5.9
  kernelVersion: 5.13.0-28-generic
  kubeProxyVersion: v1.23.3
  kubeletVersion: v1.23.3
  machineID: 9384e2927f544209b5d7b67474bbf92b
  operatingSystem: linux
  osImage: Ubuntu 21.10
  systemUUID: aa829ca9-73d7-064d-9019-df07404ad448

```

## Looking at logs

For now, digging deeper into the cluster requires logging into the relevant machines. Here are the locations of the relevant log files. On systemd-based systems, you may need to use `journalctl` instead of examining log files.

## Control Plane nodes

- `/var/log/kube-apiserver.log` - API Server, responsible for serving the API

- `/var/log/kube-scheduler.log` - Scheduler, responsible for making scheduling decisions
- `/var/log/kube-controller-manager.log` - a component that runs most Kubernetes built-in controllers, with the notable exception of scheduling (the kube-scheduler handles scheduling).

## Worker Nodes

- `/var/log/kubelet.log` - logs from the kubelet, responsible for running containers on the node
- `/var/log/kube-proxy.log` - logs from kube-proxy , which is responsible for directing traffic to Service endpoints

## Cluster failure modes

This is an incomplete list of things that could go wrong, and how to adjust your cluster setup to mitigate the problems.

## Contributing causes

- VM(s) shutdown
- Network partition within cluster, or between cluster and users
- Crashes in Kubernetes software
- Data loss or unavailability of persistent storage (e.g. GCE PD or AWS EBS volume)
- Operator error, for example, misconfigured Kubernetes software or application software

## Specific scenarios

- API server VM shutdown or apiserver crashing
  - Results
    - unable to stop, update, or start new pods, services, replication controller
    - existing pods and services should continue to work normally unless they depend on the Kubernetes API
- API server backing storage lost
  - Results
    - the kube-apiserver component fails to start successfully and become healthy
    - kubelets will not be able to reach it but will continue to run the same pods and provide the same service proxying

- manual recovery or recreation of apiserver state necessary before apiserver is restarted
- Supporting services (node controller, replication controller manager, scheduler, etc) VM shutdown or crashes
  - currently those are colocated with the apiserver, and their unavailability has similar consequences as apiserver
  - in future, these will be replicated as well and may not be co-located
  - they do not have their own persistent state
- Individual node (VM or physical machine) shuts down
  - Results
    - pods on that Node stop running
- Network partition
  - Results
    - partition A thinks the nodes in partition B are down; partition B thinks the apiserver is down. (Assuming the master VM ends up in partition A.)
- Kubelet software fault
  - Results
    - crashing kubelet cannot start new pods on the node
    - kubelet might delete the pods or not
    - node marked unhealthy
    - replication controllers start new pods elsewhere
- Cluster operator error
  - Results
    - loss of pods, services, etc
    - lost of apiserver backing store
    - users unable to read API
    - etc.

## Mitigations

- Action: Use the IaaS provider's automatic VM restarting feature for IaaS VMs
  - Mitigates: Apiserver VM shutdown or apiserver crashing
  - Mitigates: Supporting services VM shutdown or crashes
- Action: Use IaaS providers reliable storage (e.g. GCE PD or AWS EBS volume) for VMs with apiserver+etcd
  - Mitigates: Apiserver backing storage lost
- Action: Use [high-availability](#) configuration

- Mitigates: Control plane node shutdown or control plane components (scheduler, API server, controller-manager) crashing
  - Will tolerate one or more simultaneous node or component failures
- Mitigates: API server backing storage (i.e., etcd's data directory) lost
  - Assumes HA (highly-available) etcd configuration
- Action: Snapshot apiserver PDs/EBS-volumes periodically
  - Mitigates: Apiserver backing storage lost
  - Mitigates: Some cases of operator error
  - Mitigates: Some cases of Kubernetes software fault
- Action: use replication controller and services in front of pods
  - Mitigates: Node shutdown
  - Mitigates: Kubelet software fault
- Action: applications (containers) designed to tolerate unexpected restarts
  - Mitigates: Node shutdown
  - Mitigates: Kubelet software fault

## What's next

- Learn about the metrics available in the [Resource Metrics Pipeline](#)
- Discover additional tools for [monitoring resource usage](#)
- Use Node Problem Detector to [monitor node health](#)
- Use `kubectl debug node` to [debug Kubernetes nodes](#)
- Use `crictl` to [debug Kubernetes nodes](#)
- Get more information about [Kubernetes auditing](#)
- Use `telepresence` to [develop and debug services locally](#)

## Feedback

Was this page helpful?

Yes

No

---

Last modified July 04, 2025 at 11:28 PM PST: [Add documentation for graduating the NodeLocalCRISocket to Beta \(0135b1b08b\)](#)